

Reviewer Pack — Minimal Rank of Geometric Loci vs Monotone k-CLIQUE Circuit ...

Entry 91ca137f15f0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 17:09:21 UTC

Minimal Rank of Geometric Loci vs Monotone k-CLIQUE Circuit Lower Bounds

Entry ID: 91ca137f15f0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-25 17:09:21 UTC

1. Conjecture

Field A (mathematical branch): Arakelov geometry **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity for k-CLIQUE

Statement:

[‘For any instance of the monotone k-CLIQUE problem with n variables, the minimal rank of its associated Arakelov geometric loci is at least $\alpha n^{3/4}$, where α is a positive constant.’, ‘This implies that monotone circuits for k-CLIQUE require size $\Omega(n^{3/4})$.’, ‘The conjecture holds for all $n \leq 40$ within the time constraints.’]

Rationale (proposer’s reasoning):

[‘Arakelov geometry has been used to study complexity of certain polynomial equations, which may provide insights into monotone circuits.’, ‘The geometric loci can encode structural properties of the circuit that are relevant to its size.’, ‘If this conjecture is true, it would offer a new approach to proving lower bounds for monotone k-CLIQUE.’]

Taxonomy category: MONOTONE_CLIQUE (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 1582ce15f40627ca

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Minimal rank of Arakelov geometric loci is at least $\alpha n^{3/4}$ for all $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot row
        max_row = i
        for r in range(i+1, rows):
            if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                max_row = r
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below the pivot
        factor = Fraction(1, matrix[i][i])
        for j in range(i, cols):
            matrix[i][j] *= factor

        for r in range(rows):
            if r != i:
                factor = matrix[r][i]
                for j in range(i, cols):
                    matrix[r][j] -= factor * matrix[i][j]
    return matrix

def rank(matrix):
    rows, cols = len(matrix), len(matrix[0])
    rref = gaussian_elimination(matrix)
    rank_value = 0
    for row in rref:
        if any(row):
            rank_value += 1
    return rank_value

def generate_monotone_kclique(n, k):
    # This is a placeholder function. In practice, you would need to implement
    # the generation of random monotone k-CLIQUE instances.
```

```

loci = [[random.choice([0, 1]) for _ in range(k)] for _ in range(n)]
return loci

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    alpha = Fraction(1, 2) # Example constant
    results = []

    for n in n_values:
        loci = generate_monotone_kclique(n, k=n)
        rank_value = rank(loci)

        if rank_value < alpha * n**(3/4):
            return {
                "metric_name": "Minimal Rank",
                "metric_value": rank_value,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, rank_value={rank_value} < {alpha * n**(3/4)}"
            }

        results.append(rank_value)

    return {
        "metric_name": "Minimal Rank",
        "metric_value": sum(results) / len(results),
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{\"seed\": {seed}, ...run_trial output...}}")
        results.append(result)

    total_metric_value = sum(r["metric_value"] for r in results if "metric_value" in r)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, r in zip(seeds, results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"n={n}, rank_value<{alpha * n**(3/4)}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44bfc50a.py", line 91, in <module>
    result = run_trial(seed)
            ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44bfc50a.py", line 64, in run_trial
    rank_value = rank(loci)
                ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44bfc50a.py", line 42, in rank
    rref = gaussian_elimination(matrix)
          ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44bfc50a.py", line 29, in gaussian_elim
    factor = Fraction(1, matrix[i][i])
            ~~~~~
```

```
File "/usr/lib/python3.12/fractions.py", line 281, in __new__
    raise ZeroDivisionError('Fraction(%s, 0)' % numerator)
```

ZeroDivisionError: Fraction(1, 0)

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture's validity. | next: Investigate and fix the crash in the test code to continue testing the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11577
2	propose	ollama_remote	glm4:latest	0	0	11079
3	propose	ollama_remote	glm4:latest	0	0	10142
4	preregistration	ollama_remote	glm4:latest	0	0	5298
5	novelty	ollama_remote	glm4:latest	0	0	5019
6	novelty	ollama_remote	glm4:latest	0	0	5108
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21791
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11887
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10877

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10380
11	judge	ollama_remote	glm4:latest	0	0	8281

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 111439 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/91ca137f15f0.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/91ca137f15f0.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/91ca137f15f0.tar.gz (if generated)